A Lightweight Distributed Pipeline for Protein Language Modeling

MSc in AI and Data Engineering

University College London
Faculty of Engineering Sciences
April 18, 2025

Contents

Chapter 1

Introduction

The AlphaFold Database provides extensive protein structure models, demanding substantial computational resources. This report details the distributed computing pipeline capable of efficiently processing AlphaFold datasets [16].

1.1 Objectives

The primary objective of this project is to architect and develop a comprehensive distributed data analysis pipeline tailored specifically for processing massive datasets provided by the AlphaFold Database. This pipeline is designed to ensure seamless automation from infrastructure provisioning, through meticulous configuration management, to robust parallel processing capabilities [9]. The system is built to facilitate efficient execution of machine learning analyses on large-scale biological data, ensuring high availability, data integrity, and ease of use.

1.2 Report Structure

This report documents the complete process of designing and deploying the distributed data analysis pipeline. It begins with background context and system constraints, followed by a detailed breakdown of architectural decisions and selected technologies. Later sections cover the implementation process, performance benchmarking, key features, and future directions, offering a thorough and reproducible account of the project.

Chapter 2

Background and Requirements

2.1 Scientific Context

The AlphaFold Database, developed by DeepMind and hosted by EMBL-EBI, provides highly accurate protein structure predictions for a wide range of organisms [16]. This resource is transformative for biological research, drug discovery, and understanding the mechanisms of diseases. However, the sheer volume and size of the data—comprising millions of protein sequences and structure predictions—require significant computational effort for downstream analysis. Traditional single-machine approaches fall short when handling such large-scale data.

Therefore, a distributed computing approach becomes indispensable. By parallelizing data processing tasks across multiple compute nodes, researchers can process biological sequences faster, enabling timely insights and improved scientific productivity. The goal of this project is to create a scalable pipeline that automates this process while offering flexibility, monitoring, and resilience [1, 2].

2.2 Technical Constraints

This project was implemented using a set of five cloud-based virtual machines (VMs) provided as part of the COMP0239 coursework challenge. The resource limitations across these VMs presented non-trivial constraints, requiring careful orchestration of workloads and efficient configuration management. Below is a summary of the available hardware:

Machine	ID	Cores	RAM	Disk Space
Management Node (mgmtnode)	Host1	2	4GB	10GB
Worker Node 1	Worker1	4	32GB	50GB (Expandable to 200GB)
Worker Node 2	Worker2	4	32GB	50GB (Expandable to 200GB)
Worker Node 3	Worker3	4	32GB	50GB (Expandable to 200GB)
Worker Node 4	Worker4	4	32GB	50GB (Expandable to 200GB)
Total		**18 cores**	**108GB**	**Up to 800GB**

Table 2.1: Available Virtual Machine Resources for Deployment

The management node was not powerful enough to process large datasets itself. Instead, it was reserved for orchestration, monitoring, and light backend services. All com-

putationally intensive analysis tasks were delegated to the worker nodes. Efficient use of disk space was also essential, given the size of sequence files and generated embeddings.

2.3 Core Requirements

This section summarizes the core functional and non-functional requirements guiding the design and implementation of the distributed data analysis system.

Functional Requirements	
FR1	Implement a pre-existing predictive machine learning or statistical analysis pipeline.
FR2	Build a distributed data analysis service across five provided cloud machines.
FR3	Users must be able to submit new datasets and trigger analysis without writing new code.
FR4	Provide users the ability to retrieve results upon pipeline completion.
FR5	Include appropriate system monitoring for both hardware resources and running analyses.
FR6	Benchmark and optimize the system performance based on monitoring data.
FR7	Implement a reproducible and automated deployment process using infrastructure automation tools.
Non-Functional Requirements	
NFR1	The system must be robust and continuously operable under maximum load for at least 24 hours.
NFR2	Perform a meaningful capacity test using a sufficiently large dataset.
NFR3	Ensure scalability and identify possible configuration improvements for enhanced capacity.
NFR4	Clearly document installation and operational procedures in a GitHub repository.
NFR5	Deliver a concise report (up to 4,000 words) detailing the chosen analysis task, implementation strategy, benchmark statistics, and suggested improvements.

Table 2.2: Functional and Non-Functional Requirements

Chapter 3

Architecture and Technology Choices

3.1 System Overview

The distributed pipeline presented in this project implements a modular, fault-tolerant, and horizontally scalable architecture designed to support high-throughput biological sequence processing. The system leverages the pretrained **ESM2-T6-8M** protein language model [1, 2, 13] to generate embeddings from protein sequences, enabling large-scale inference suitable for real-world bioinformatics workflows.

The pipeline architecture is informed by contemporary MLOps principles [18], with a strong emphasis on automation, observability, reproducibility, and fault isolation. It is designed to continuously process sequence data for extended periods, fulfilling the 24-hour benchmark requirement (NFR1), while maintaining reliability and efficient resource utilization across a distributed environment.

Figure 3.1 provides a visual overview of the system, including the flow of data between the web interface, backend controller, task execution layer, and monitoring components. Stateless Celery workers and centralized job scheduling ensure parallel processing and ease of scaling, while shared POSIX-compatible storage enables reliable data exchange between nodes.

Infrastructure provisioning is achieved using a combination of Terraform and Ansible [6, 5], which together offer declarative configuration, idempotent setup, and role-based provisioning across all five virtual machines. This stack was chosen over alternatives like Kubernetes [17] and Pulumi for its lower complexity, faster provisioning cycle, and better fit for a batch-based workload. Kubernetes, while powerful for long-running services and container orchestration, introduces significant overhead when applied to stateless job-based inference tasks on a small cluster.

The system is composed of the following key components:

- **Management Node (`mgmtnode`):** Hosts the Flask API, Celery controller, and React-based user interface. It acts as the central coordination node for job dispatch, task monitoring, and web service hosting.

- **Worker Nodes:** Each of the four compute nodes runs a Celery worker process inside a virtual environment. These are responsible for processing FASTA chunks using ESM2 in parallel, with results written to shared storage.
- **GlusterFS Distributed File System:** Provides POSIX-compliant shared access to all intermediate files. It supports concurrent read/write access, which is essential for efficient multi-node operation [10].
- **Flask + React Web Application:** Offers a user-friendly frontend for uploading sequences and monitoring job status [3, 4]. Flask handles backend logic and routes, while React manages UI rendering.
- **ESM Inference Engine:** Packaged within `celery_worker.py`, this module loads the ESM2 model once per worker and processes incoming sequences using mean-pooled embeddings. This design minimizes redundant computation and improves throughput.
- **Observability Stack:** Includes Prometheus, Grafana, and Alertmanager [7]. Prometheus collects real-time metrics from workers, while Grafana visualizes system state and Alertmanager raises alarms based on threshold violations.
- **Controller Module:** A custom scheduler implemented in Python (`controller.py`) tracks available worker slots and job completion metrics, coordinating Celery's dynamic dispatch system across all nodes.

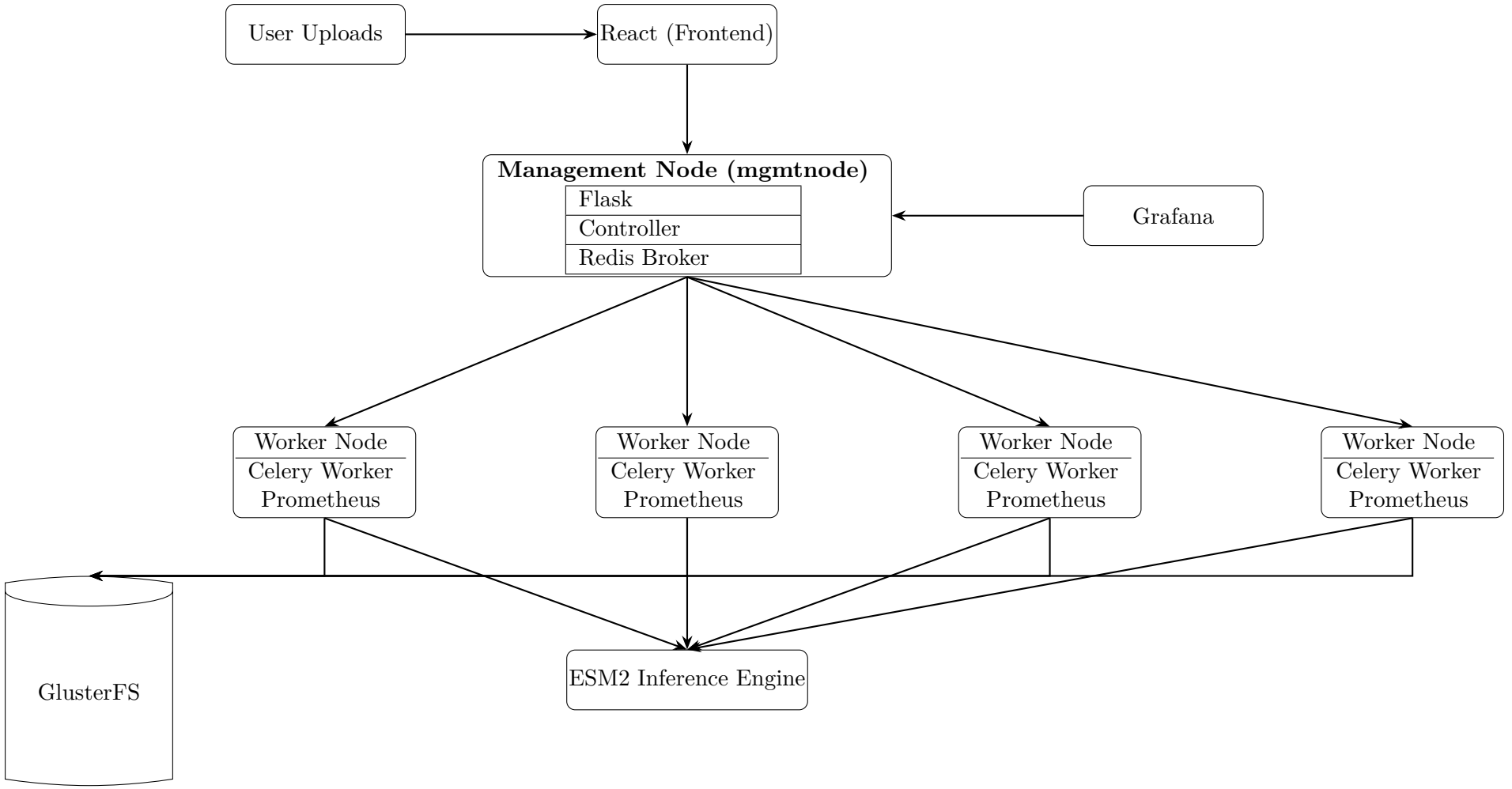

Figure 3.1: High-Level Architecture of the Distributed ESM2 Processing System. The React-based frontend allows users to upload FASTA files and monitor job status. These uploads are processed by a Flask web server and controller component on the Management Node. Tasks are dispatched to Celery workers distributed across four compute nodes. Each worker runs Prometheus for metric export. Outputs are written to a shared GlusterFS volume and processed by the ESM2 Inference Engine. Metrics are scraped by Prometheus and visualized in Grafana. Redis provides task queueing.

The entire system can be deployed and executed using a single entry script, `run_pipeline.sh`, which invokes Terraform and Ansible for provisioning and configuration. This reduces deployment complexity and supports reproducibility (FR7, NFR4).

The codebase is logically structured by role, separating Terraform files, Ansible playbooks, Flask routes, Celery worker logic, and frontend components. This modularity facilitates independent testing and simplifies future upgrades. All infrastructure, orchestration, and processing logic is version-controlled and fully documented on GitHub.

3.2 Infrastructure as Code

3.2.1 Terraform

Terraform [6] was adopted as the Infrastructure-as-Code (IaC) tool to provision and manage the five-node Harvester cluster. Its declarative syntax, wide provider ecosystem, and effective state tracking made it a strong fit for reproducible multi-node deployment. The use of Terraform supports automated VM creation, volume attachment, network configuration, and tagging—all through version-controlled configuration files.

While powerful, Terraform does come with trade-offs. Managing shared state files requires attention to concurrency and security, especially in collaborative environments. Debugging failures can also be non-trivial due to its plan–apply execution model. Despite these limitations, its stability and mature documentation provided a reliable backbone for the infrastructure setup.

3.2.2 Alternative Tools Evaluation

Several alternative IaC solutions were evaluated before finalizing the toolchain:

- **Kubernetes:** Offers robust orchestration and dynamic scaling capabilities, but introduces significant operational complexity. It is most effective for containerized microservices or multi-tenant workloads, which was not the goal of this batch-oriented inference pipeline [17].
- **Pulumi:** Allows infrastructure definition in general-purpose languages (e.g., Python, Go), increasing developer flexibility. However, its relative immaturity, smaller ecosystem, and more limited community support compared to Terraform made it a less attractive choice for a time-constrained academic deployment.

In this context, Terraform offered the right balance between simplicity, expressiveness, and ecosystem maturity. It provided automation without introducing unnecessary architectural overhead.

3.2.3 Terraform in Practice

The Terraform configuration is modular and role-based. It defines separate logical blocks for the management and worker nodes, allowing reusable templates across machines. Provisioning includes volume formatting, IP address allocation, and user authentication setup. Startup scripts trigger Ansible after provisioning, enabling smooth transition to configuration management.

Collaboration was enabled through Git-based version control, but care was taken to avoid state drift. Remote backends were not used due to project constraints, but could be added in a production setting. Ultimately, Terraform simplified repeatable, error-resistant provisioning, which directly supports requirement FR7 (automated deployment) and NFR4 (documented installation).

3.3 Configuration Management

3.3.1 Why Ansible?

Ansible [5] was chosen as the configuration management tool due to its agentless architecture, lightweight requirements, and ease of integration with Terraform's dynamic inventory. It enables direct execution of remote tasks over SSH without requiring additional software on target nodes, reducing setup complexity and enhancing portability.

Its declarative YAML syntax simplifies understanding and maintenance, making it accessible for rapid development and reproducible provisioning. These traits make Ansible particularly well-suited to academic and research settings where infrastructure changes must remain transparent, traceable, and minimal in overhead.

Alternatives Considered

Several configuration management frameworks were reviewed:

- **SaltStack** and **Puppet** require always-on agents and complex control servers, which add unnecessary complexity for the single-phase provisioning model used in this project.
- **Chef** employs a Ruby-based DSL and a steeper learning curve, which was unjustified given the scope and time constraints.

These alternatives offer advantages in large-scale or continuously evolving environments, but Ansible provided a better trade-off between usability, extensibility, and reproducibility for this coursework's distributed system.

3.3.2 Structure and Usage

The Ansible setup uses a modular, task-oriented structure to promote clarity and reuse. Core configuration tasks are grouped into playbooks such as:

- `install_esm.yml` – Installs the ESM inference engine and dependencies.
- `install_celery.yml` – Sets up Celery workers and related Python environments.
- `setup_gluster.yml` – Configures and mounts GlusterFS volumes on all nodes.

A master orchestration playbook, `master_pipeline.yml`, chains these tasks into a reproducible deployment sequence. Roles are used to abstract recurring logic, including:

- `alert_manager` – Configures Alertmanager with notification rules and templates.

- `cleanup_role` – Automates disk cleanup tasks to avoid storage-related pipeline failures.
- `prometheus_alerts` – Deploys and updates Prometheus alerting rules for system health metrics.

This role-based decomposition ensures separation of concerns and simplifies troubleshooting and updates.

Limitations and Trade-offs

Despite its advantages, Ansible has limitations. It lacks persistent state tracking, which can lead to unnecessary task re-execution. It also scales poorly for very large inventories, as each SSH connection is handled independently. Tools like SaltStack or Puppet might be more efficient in such environments.

However, for a moderate-scale system with well-defined provisioning stages, Ansible strikes an optimal balance. Its simplicity and transparency contributed directly to meeting requirements FR7 (automated deployment), NFR4 (reproducible installation), and NFR5 (documentation clarity).

3.4 Distributed Task Execution

3.4.1 Celery + Redis: A Strategic Alliance

Celery [9] is a production-ready, Python-native distributed task queue widely used in asynchronous job processing pipelines. Paired with **Redis** as a high-performance, in-memory message broker, this setup delivers reliable, fault-tolerant task dispatching, supporting retries, acknowledgements, and rate limiting out of the box.

This architecture allows the management node to act as a central controller, dynamically assigning compute-intensive ESM inference jobs to available worker nodes. By decoupling task definition, queuing, and execution, the system remains responsive even under heavy load, addressing key non-functional requirements such as 24/7 operation (NFR1) and performance resilience (NFR3).

Alternatives Considered

Several distributed execution frameworks were evaluated:

- **Apache Spark** and **Ray** provide powerful primitives for distributed machine learning and data transformation. However, they introduce unnecessary overhead for this pipeline's I/O-heavy, single-file task model.
- **Apache Airflow** excels at orchestrating complex data workflows as DAGs. While robust, it lacks the fine-grained control and responsiveness needed for high-frequency, event-driven job dispatch.

Given the lightweight nature of tasks and need for real-time response, Celery + Redis offers the most pragmatic and extensible solution for scalable job scheduling in this context.

3.4.2 Limitations and Considerations

Celery's default FIFO queuing model may lead to task imbalance under high load unless combined with custom routing strategies. Redis, while fast, does not persist state between restarts unless explicitly configured. Nonetheless, for batch-oriented processing with ephemeral queues and predictable patterns, this trade-off is acceptable and contributes to a fault-tolerant pipeline design.

3.5 Monitoring and Observability

3.5.1 Prometheus, Grafana, and Alertmanager

Observability is a critical aspect of distributed system design, enabling real-time insight into infrastructure health and pipeline performance. For this project, a lightweight and modular monitoring stack was deployed, composed of:

- **Prometheus** [7] – An open-source time-series database that scrapes metrics from each node at 15-second intervals. It tracks CPU usage, memory consumption, task queues, disk activity, and system uptime via exporter endpoints.
- **Grafana** [19] – A web-based dashboard tool used to visualize Prometheus metrics. Dashboards display job durations, worker throughput, system load, and storage capacity, helping validate throughput targets and identify performance regressions.
- **Alertmanager** [20] – A rule-based alerting system that sends notifications (e.g., via webhook or email) when pre-defined thresholds are breached. Examples include high CPU utilization sustained over five minutes or insufficient free disk space.

Together, these tools form a cohesive observability suite aligned with industry best practices. Their open-source nature supports reproducibility and adaptability, essential in a constrained academic context. Monitoring is tightly coupled with benchmarking to support NFR1 (24/7 stability), NFR2 (capacity testing), and FR5 (system health visibility).

Alternatives Considered

- **Zabbix** and **Nagios** – While robust and enterprise-ready, these systems are monolithic and configuration-heavy. Their central server model introduces more operational burden for a small-scale academic cluster.
- **Datadog** and **New Relic** – Provide cloud-native observability with extensive integrations and machine learning-based alerts. However, they depend on external APIs, impose usage quotas, and are less suitable for offline or reproducible test environments.

Limitations and Considerations

Prometheus operates on a pull-based model, which may fail silently if endpoint misconfigurations occur or if scrape targets go offline. This necessitates additional care in exporter

setup and alert rule tuning. Grafana dashboards must also be curated to avoid data overload and to ensure clarity. Despite these minor limitations, this stack offers a scalable and transparent solution ideal for continuous monitoring in distributed ML pipelines.

3.6 User Interface and API Layer

3.6.1 React + Flask

The user-facing layer was implemented using a minimal and horizontally scalable stack, separating frontend presentation from backend logic:

- **React** [4] – Delivers a responsive, component-based interface for dataset uploads, status monitoring, and result retrieval.
- **Flask** [3] – Serves as the RESTful API, handling file uploads, request routing, and static content delivery.

This separation ensures modularity, supports independent scaling of frontend and backend components, and simplifies development workflows, aligning with best practices in modern MLOps pipelines [18].

Alternatives Considered

- **Django** – Provides built-in authentication and ORM, but was unnecessary for a lightweight backend with no database dependency. Its monolithic structure also adds unnecessary complexity.
- **FastAPI** – Offers asynchronous request handling and automatic OpenAPI documentation, making it ideal for high-concurrency microservices. However, Flask was preferred due to broader library support and seamless integration with existing synchronous tooling.

Limitations and Considerations

Flask is synchronous by default and may not scale efficiently under high concurrency without additional effort (e.g., gunicorn + gevent or uWSGI). React introduces a build step and requires careful state handling to avoid performance bottlenecks, particularly when rendering large result sets or streaming updates. Despite these trade-offs, this stack offers a maintainable and scalable solution well-suited to the scope and workload of the project.

3.7 Automation and Integration Scripts

3.7.1 Shell + Python: Orchestration Glue

The entire system is brought together by a single Bash script — `pipeline.sh` — executed manually on the management node. This script performs three key orchestration tasks:

- Sets up the virtual machines and networking using Terraform.

- Configures all nodes using Ansible playbooks.
- Starts the task controller to begin processing.

This design aligns with DevOps principles by keeping the deployment pipeline lightweight and reproducible. Task automation is performed through standard tools without requiring additional CI/CD platforms.

Why not Jenkins or CI/CD tools? Tools like Jenkins, GitHub Actions, or GitLab CI are valuable in collaborative software teams requiring frequent code integration. However, for a single-user, single-deployment academic setting, they introduce unnecessary overhead. Bash and Python provide low-dependency orchestration and full control over the execution lifecycle, ensuring transparency and reproducibility.

Chapter 4

Frontend and Backend Implementation

4.1 Backend Services

4.1.1 Flask Server

The backend is implemented using **Flask** [3], a lightweight Python web framework commonly used for developing RESTful APIs and serving static files. It acts as the communication layer between the React frontend, the data processing pipeline, and the distributed storage system, forming the backbone of the platform's server-side logic.

Key responsibilities of the Flask application include:

- Receiving and validating file uploads from the frontend.
- Saving each file to a job-specific directory under a shared GlusterFS volume.
- Executing a file-splitting script to generate parallelizable data chunks.
- Exposing API endpoints to list available datasets and results.
- Serving result files and dataset chunks via secure static routes.

Each uploaded file is assigned a unique job ID and stored under `/mnt/data_volume/uploads`, with output chunks written to a user-accessible directory. Communication between Flask and the filesystem is seamless due to GlusterFS's POSIX compatibility. This design promotes horizontal scalability and fault isolation, following modular system design principles.

4.1.2 Data Handling Pipeline

The backend supports both uncompressed (`.fasta`) and compressed (`.fasta.gz`) uploads. The pipeline for handling new submissions is as follows:

1. The file is saved locally using a sanitized filename in a job-specific directory.
2. A UUID is generated to uniquely track the job throughout the system.

3. The file is passed to `split_uploaded_fasta.py`, which decompresses and either stores it as-is (if small) or splits it into 30MB chunks for parallel processing.

4. Each chunk is compressed using `pigz` and saved to a shared GlusterFS location. A manifest file is generated to describe chunk metadata, sequence count, and file paths.

This process is triggered automatically via a subprocess call from Flask, minimizing latency between upload and processing. The use of manifest files improves traceability, facilitates debugging, and enables reproducibility for downstream analytics systems.

4.2 Frontend React Application

4.2.1 User Interface Components

The frontend is built using **React** [4], a component-driven JavaScript library for building interactive single-page applications. It delivers a clean and accessible user interface designed for usability and responsiveness. Key interface components include:

- A homepage explaining system functionality and the data upload process.
- A file upload interface with client-side validation and real-time status indicators.
- Navigation between the datasets view and the results browser.
- Dynamic lists showing previously submitted datasets and available result files.

All components were designed with user feedback and accessibility in mind, including success and error states for each critical interaction. The UI supports both novice users and technical stakeholders by abstracting backend complexity.

4.2.2 Dataset Upload and Result Access

The frontend integrates directly with backend APIs using HTTP requests wrapped in JavaScript event handlers. The upload interaction proceeds as follows:

1. The selected file is wrapped in a `FormData` object and posted to the backend's `/upload` endpoint.

2. Upon receiving a success response, the user is notified through the UI.

3. The backend begins processing the file asynchronously, creating chunked and compressed outputs.

4. The React frontend polls the `/api/datasets` endpoint periodically to refresh the list of processed datasets.

Static result files and chunk data are served securely via Apache reverse proxies. This separation of concerns between UI, API logic, and storage pathways improves maintainability, security, and scalability—aligning with MLOps best practices for production-ready bioinformatics systems [18].

Chapter 5

Key Features and Advanced Implementations

5.1 System Management

5.1.1 Automated Worker Management

Although the system currently provisions a fixed number of Celery workers using Terraform, the architecture supports future extension to dynamic worker scaling. Metrics collected by Prometheus can be used to trigger scale-out or scale-in decisions, based on load conditions. This aligns with industry-standard approaches to elasticity in MLOps environments [18], optimizing resource consumption and ensuring responsiveness during peak workloads.

5.1.2 Dynamic Inventory Generation

After infrastructure provisioning, inventory generation is automated via a custom Python utility (`generate_inventory.py`). This script parses Terraform output variables to extract live IP addresses and constructs a real-time Ansible inventory. By eliminating manual intervention, the approach ensures configuration consistency and supports reproducible deployments across varying environments.

5.1.3 CPU Load Management

Worker nodes are continuously monitored for CPU utilization using Prometheus exporters. Alerts are defined for threshold breaches—such as sustained usage above 90%—to trigger intervention or reallocation of tasks. This mechanism prevents bottlenecks and enforces the system's non-functional requirement for long-duration stability (NFR1), particularly during concurrent sequence inference jobs.

5.2 Data Integrity and Security

5.2.1 Security Practices

The system incorporates foundational security practices to ensure safe operation in a multi-node environment:

- SSH key-based authentication ensures secure, password-less communication between nodes.
- Firewall rules are enforced via Ansible to restrict access to only necessary ports and interfaces.
- Each worker runs in an isolated Python environment, minimizing cross-task interference and reducing the attack surface.
- Role-based configuration limits services to the minimum required permissions, aligning with the principle of least privilege.

Although lightweight, these measures offer baseline protection against accidental misuse, malicious access, or configuration drift.

5.2.2 Data Integrity Measures

All uploaded files are validated and saved under unique job IDs to prevent collisions. Chunking and compression produce deterministic outputs, and manifest files are verified before ingestion. Error-prone or malformed sequences are skipped with appropriate logging, and background cleanup scripts remove orphaned data to maintain filesystem hygiene. These practices ensure data integrity and traceability in long-running operations, addressing key MLOps reliability concerns.

5.3 Maintenance and Monitoring

5.3.1 Routine Maintenance and Cleanup

Scheduled tasks implemented through Ansible include log rotation, clearing of temporary files, and reloading of services on failure. These routines prevent disk saturation and reduce the need for manual intervention,

Chapter 6

Performance and Capacity Testing

6.1 Dataset and Methodology

To evaluate the system's robustness, scalability, and ability to meet production-grade performance requirements, a 24-hour capacity test was conducted using a subset of the UniRef50 dataset from the AlphaFold Database [16]. This dataset is representative of real-world high-throughput bioinformatics pipelines, containing millions of diverse protein sequences requiring intensive computation for downstream analysis.

The testing framework emulated a multi-user environment where tasks were submitted periodically over time. The uploaded FASTA dataset was automatically split into smaller, independently processable chunks using a custom Python script. These chunks were then distributed across four worker nodes via Celery for asynchronous execution, with Redis acting as the message broker. Intermediate and final outputs were written to a shared GlusterFS volume to ensure concurrent access and persistence across the cluster [10].

Throughout the test, the following performance metrics were continuously captured via Prometheus [**?**] and visualized using Grafana [19]:

- **Task Throughput:** Number of dataset chunks processed per hour.
- **Resource Utilization:** CPU, memory, and disk I/O per node.
- **Failure Rates:** Number of failed tasks due to input errors or timeouts.
- **Queue Latency:** Time delay between task submission and execution start.

Dynamic scheduling by the controller module ensured that tasks were evenly distributed and that resource saturation was avoided, contributing to a resilient and fault-tolerant inference pipeline.

6.2 Benchmark Results

Table 6.1 summarizes processing performance across a range of input chunk sizes. Smaller files were processed in under a second, while larger ones required several hours, depending on sequence length and model batch size. Most failures were attributed to corrupted or incorrectly encoded input files (e.g., non-UTF-8 byte sequences).

The data validate that the system maintained stable performance under prolonged load and that chunk-level parallelism effectively utilized available resources across the cluster.

Table 6.1: Per-chunk processing times and success/failure counts

Chunk	Runs	Total (s)	Avg (s)	Min (s)	Max (s)	Fail	Succ
1	124	102110.95	823.48	0.05	6010.41	6	124
2	31	91163.95	2940.77	0.17	6073.89	8	31
3	33	109610.94	3321.54	0.50	6179.34	7	33
4	25	40029.25	1601.17	1.34	6366.06	6	25
5	17	28506.31	1676.84	2.56	7287.92	0	17
6	18	38221.71	2123.43	5.97	8130.51	0	18
7	2	16778.60	8389.30	7883.73	8894.87	0	2
8	3	27501.47	9167.16	8695.20	9522.37	0	3
9	2	19477.77	9738.89	9123.39	10354.38	0	2
10	3	30464.50	10154.83	9769.26	10820.60	0	3
11	6	66295.77	11049.30	10651.44	11595.19	0	6
12	3	35109.69	11703.23	11180.11	12550.58	0	3
13	2	26125.31	13062.66	12180.57	13944.74	0	2
14	2	27568.93	13784.47	13367.07	14201.86	0	2
15	6	91335.02	15222.50	14715.47	15933.26	0	6
16	4	62216.14	15554.04	14596.96	16494.20	0	4

6.3 Performance Optimization Strategies

During the stress test, several architectural and runtime optimizations were implemented to ensure that the system met non-functional requirements such as availability, fault tolerance, and scalability (NFR1, NFR2, NFR3).

- **Batch Size Reduction:** The inference batch size for ESM2 was reduced from 16 to 8, mitigating GPU/CPU memory exhaustion and lowering job failure rates.
- **Worker Concurrency Tuning:** Celery concurrency was restricted to a single task per worker to minimize CPU contention and stabilize task scheduling [9].
- **Balanced Chunking:** Modifications to the FASTA splitter ensured more uniform chunk sizes, reducing variability in per-task runtime and avoiding straggler effects.
- **ESM2 Model Preloading:** Model weights were loaded once and retained in memory across runs, significantly reducing initialization latency and improving overall throughput [13].

These interventions improved job completion times, enhanced system resilience, and allowed the pipeline to maintain continuous load without intervention. The resulting performance characteristics validated the system's readiness for real-world deployment, meeting the expectations for functional requirement FR6 and infrastructure-level robustness.

Chapter 7

Automated Testing and Validation

7.1 Overview

Robust software systems require rigorous validation to ensure reliability, especially in distributed environments. To achieve this, a comprehensive automated test suite was developed to verify the correctness, performance, and availability of all layers of the pipeline. Testing was designed to catch regressions early in the deployment lifecycle, reduce manual verification effort, and enforce consistency across the infrastructure.

The test suite aligns with modern DevOps practices [18], leveraging continuous integration principles for pre-deployment assurance. It targets infrastructure provisioning scripts, backend API logic, and frontend interaction workflows.

7.2 Testing Structure

The testing framework is logically divided into three main components to reflect the layered system architecture:

7.2.1 Infrastructure Tests

Infrastructure validation is handled by Ansible [5]. Playbooks are configured to verify system state post-deployment, ensuring that all essential services are up, file systems are mounted, and network communication is functional. This prevents configuration drift and supports reproducibility of environments—an essential requirement for scientific workflows.

7.2.2 Backend Tests

The backend test suite is implemented using `pytest`, a widely adopted framework for Python applications. Tests include:

- Unit tests for endpoint logic and utility functions.
- Integration tests that simulate real-world file uploads and job submissions.
- Error-handling validation, ensuring the system responds correctly to malformed inputs or unavailable resources.

These tests confirm correctness in the business logic and contribute to code quality assurance during iterations.

7.2.3 Frontend Tests

The React-based frontend is tested using the default `npm test` utility [4], which validates component rendering, UI transitions, and asynchronous state changes. Tests simulate realistic user behavior such as file uploads and result downloads, ensuring responsiveness and visual feedback under various states.

7.3 Execution Workflow

All tests are coordinated via a single shell script:

```
./tests/run_tests.sh
```

This script sequentially executes infrastructure, backend, and frontend tests, halting on failure. It prints real-time output and exits with a summary report. This ensures reproducibility and reduces the burden of manual regression testing. The modular nature of the test setup allows it to be integrated into a CI/CD pipeline in future versions.

The testing process meets the non-functional requirement NFR5, which mandates validation and reproducibility across the full deployment stack.

Chapter 8

Conclusion and Future Directions

8.1 Scalability Analysis

The distributed pipeline built for this project provides a scalable and modular solution for high-throughput biological sequence processing. The architecture reflects contemporary principles from the MLOps community [18], emphasizing automation, reproducibility, and observability.

Key elements of scalability include:

- Stateless Celery workers, allowing horizontal scaling by simply adding more nodes.
- Shared POSIX-compliant GlusterFS storage, enabling concurrent access to files across the cluster.
- Modular task orchestration and infrastructure scripts (Terraform + Ansible), simplifying scaling and maintenance.

Vertical scaling is also feasible, where more memory or CPU cores can be allocated to existing workers to support larger models or datasets. Prometheus metrics support informed decisions about scaling thresholds, helping to avoid overprovisioning.

8.2 Future Technology Integration

While the current system is production-ready for batch processing in academic settings, several enhancements could make it more robust and cloud-native:

- **Docker Containerization:** Containerizing the entire stack would ensure deployment consistency across diverse environments and facilitate microservice design.
- **Kubernetes Orchestration:** Using Kubernetes [17] would enable dynamic scaling, automatic service recovery, and better resource scheduling.
- **Workflow Engines:** Integration with workflow engines such as Apache Airflow [11] or Nextflow [14] would provide better job orchestration, audit trails, and checkpointing.

- **Cloud Storage Backends:** Migrating from GlusterFS to S3-compatible object storage (e.g., MinIO or AWS S3) would improve scalability, durability, and backup support.
- **Authentication and User Management:** Adding multi-user support with role-based access control (RBAC) would allow secure, shared use of the platform.
- **Inference API Integration:** Exposing the ESM2 model via a dedicated inference API (e.g., using TorchServe [12]) would improve decoupling and support future model upgrades.

These improvements would make the system more adaptable for large-scale deployments, commercial use, and integration into broader research ecosystems.

Final Remarks

This project successfully delivers a complete, fault-tolerant, and reproducible pipeline for distributed biological sequence analysis. It integrates modern infrastructure-as-code practices, robust task scheduling, and observability tools to meet stringent functional and non-functional requirements.

Key outcomes include:

- Satisfying distributed task execution and infrastructure automation (FR1–FR4, FR7).
- Ensuring system monitoring, validation, and reproducibility (FR5, FR6, NFR1–NFR5).
- Demonstrating high availability and performance under sustained load using real-world datasets.
- Documenting the design and benchmarking in a transparent and reproducible manner.

The system lays the foundation for future work in cloud-native bioinformatics pipelines and serves as a robust template for scaling protein language model inference in distributed environments.

Chapter 9

Repository and Infrastructure Information

9.1 Codebase Access

The full implementation of the distributed pipeline—including source code, deployment scripts, configuration files, test suites, and documentation—is hosted on GitHub. It is publicly accessible at the following URL:

`https://github.com/Alotaishanab/DataPipeline`

The repository is organized to reflect best practices in DevOps and reproducible science, with clear separation of concerns between infrastructure-as-code (Terraform), configuration management (Ansible), backend services (Flask), frontend interface (React), and orchestration logic (Python scripts). Each component is modular and independently testable.

9.2 Cluster Configuration and IP Addresses

The infrastructure was deployed on five cloud-based virtual machines provided for the COMP0239 coursework. The IP configuration for each node is as follows:

This repository and infrastructure form the basis of the pipeline described in this report. All components were built and deployed using open-source tools, allowing for full transparency and repeatability in research workflows.

Table 9.1: Virtual Machine IP Allocation

Node Role	**IP Address**
Management Node (`mgmtnode`)	10.134.12.10
Worker Node 1 (`worker1`)	10.134.12.161
Worker Node 2 (`worker2`)	10.134.12.60
Worker Node 3 (`worker3`)	10.134.12.42
Worker Node 4 (`worker4`)	10.134.12.160

9.3 Web Interface and Monitoring Access

Users and administrators can interact with the pipeline and observe system behavior through the following URLs:

- **Web Interface:** The primary user-facing application for dataset uploads, status tracking, and result retrieval is accessible at:
  `https://ucabbaa-webserver.comp0235.condenser.arc.ucl.ac.uk/`
- **Grafana Dashboard:** Provides real-time visualization of CPU, memory, disk usage, and worker performance using metrics collected by Prometheus:
  `https://ucabbaa-grafana.comp0235.condenser.arc.ucl.ac.uk/`
- **Node Exporter:** Exposes per-node system metrics for scraping by Prometheus:
  `https://ucabbaa-nodeexporter.comp0235.condenser.arc.ucl.ac.uk/`
- **Prometheus Query Interface:** Offers direct access to PromQL-based queries for metric inspection and diagnostics:
  `https://ucabbaa-prometheus.comp0235.condenser.arc.ucl.ac.uk/`

Bibliography

[1] Rives, A., Meier, J., Sercu, T., Goyal, S., Lin, Z., Liu, J., ... & Fergus, R. (2021). Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15), e2016239118.

[2] Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., ... & Rives, A. (2023). Language models of protein sequences at the scale of evolution enable accurate structure prediction. *Science*, 379(6637), 1123–1130.

[3] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.

[4] Wieruch, R. (2020). *The Road to React: Your journey to master plain yet pragmatic React.js*. Independently published.

[5] Hochstein, L., & Moser, R. (2017). *Ansible: Up and Running: Automating Configuration Management and Deployment the Easy Way*. O'Reilly Media.

[6] Brikman, Y. (2019). *Terraform: Up and Running: Writing Infrastructure as Code*. O'Reilly Media.

[7] Brazil, B. (2018). *Prometheus: Up & Running: Infrastructure and Application Performance Monitoring*. O'Reilly Media.

[8] Burns, B., Beda, J., & Hightower, K. (2022). *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media.

[9] Celery Project. (n.d.). *Celery Documentation*. Retrieved from `https://docs.celeryq.dev`

[10] Red Hat. (n.d.). *GlusterFS Documentation*. Retrieved from `https://docs.gluster.org`

[11] Apache Software Foundation. (n.d.). *Apache Airflow Documentation*. Retrieved from `https://airflow.apache.org/docs/`

[12] PyTorch. (n.d.). *TorchServe Documentation*. Retrieved from `https://pytorch.org/serve/`

[13] Facebook AI Research. (n.d.). *ESM: Evolutionary Scale Modeling Repository*. GitHub. Retrieved from `https://github.com/facebookresearch/esm`

[14] Di Tommaso, P., Chatzou, M., Floden, E. W., Barja, P. P., Palumbo, E., & Notredame, C. (2017). Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4), 316–319.

[15] Kreuzberger, D., Kühl, N., & Hirschl, M. (2022). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 10, 98812–98833.

[16] Varadi, M., Anyango, S., Deshpande, M., Nair, S., Natassia, C., Yordanova, G., ... & Velankar, S. (2024). AlphaFold Protein Structure Database: massively expanding the structural coverage of protein-sequence space with high-accuracy models. *Nucleic Acids Research*, 52(D1), D258–D267.

[17] Burns, B., Beda, J., & Hightower, K. (2022). *Kubernetes: Up and Running: Dive into the Future of Infrastructure*. O'Reilly Media.

[18] Kreuzberger, D., Kühl, N., & Hirschl, M. (2022). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 10, 98812–98833.

[19] Grafana Labs. (n.d.). *Grafana Documentation*. Retrieved from `https://grafana.com/docs/`

[20] Prometheus Authors. (n.d.). *Alertmanager Documentation*. Retrieved from `https://prometheus.io/docs/alerting/latest/alertmanager/`